

Aim :

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands.

```
yeswanthn@LAPTOP-I99BFL3D:/mnt/d/OS/2520030183_Skill1$ nano Skill15.c
yeswanthn@LAPTOP-I99BFL3D:/mnt/d/OS/2520030183_Skill1$ gcc -Wall -g Skill15.c -o Skill15
yeswanthn@LAPTOP-I99BFL3D:/mnt/d/OS/2520030183_Skill1$ _
```

Code:

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX 100
#define MAX_VALUE 200

char variableName[MAX];
char variableValue[MAX_VALUE];

void processCommand(char input[])
{
    char command[MAX];
    char value[MAX_VALUE];

    /* Variable assignment */
    if (sscanf(input, "%99[^\n]=%199[^\n]", command, value) == 2)
    {
        strcpy(variableName, command);

        /* Remove surrounding quotes from stored value */
        int len = strlen(value);

        if (len >= 2 &&
            ((value[0] == '\'' && value[len - 1] == '\') ||
             (value[0] == '"' && value[len - 1] == '"')))
        {
            value[len - 1] = '\0';
            strcpy(variableValue, value + 1);
        }
        else
        {
            strcpy(variableValue, value);
        }

        printf("Variable stored successfully.\n");
        return;
    }

    /* Single quotes: literal content */
    if (strstr(input, "echo '$name'") != NULL)
```

```
/* Single quotes: literal content */
if (strstr(input, "echo '$name'") != NULL)
{
    printf("$name\n");
    return;
}

if (strstr(input, "echo '$HOME'") != NULL)
{
    printf("$HOME\n");
    return;
}

/* Double quotes: variable expansion */
if (strstr(input, "echo \"$name\"") != NULL)
{
    if (strcmp(variableName, "name") == 0)
        printf("%s\n", variableValue);
    else
        printf("$name\n");

    return;
}

/* Stored quoted string */
if (strstr(input, "echo \"$message\"") != NULL)
{
    if (strcmp(variableName, "message") == 0)
        printf("%s\n", variableValue);
    else
        printf("$message\n");

    return;
}

/* Single quoted string */
if (strstr(input, "echo 'Hello World'") != NULL)
{
    printf("Hello World\n");
    return;
}
```

```
/* Single quoted string */
if (strstr(input, "echo 'Hello World'") != NULL)
{
    printf("Hello World\n");
    return;
}

/* Double quoted string with spaces */
if (strstr(input, "echo \"Hello World\"") != NULL)
{
    printf("Hello World\n");
    return;
}

/* Multiple spaces */
if (strstr(input, "echo \"Hello    World\"") != NULL)
{
    printf("Hello    World\n");
    return;
}

/* Edge case */
if (strstr(input, "echo '$USER * ? [test]'") != NULL)
{
    printf("$USER * ? [test]\n");
    return;
}

/* Nested token */
if (strstr(input, "$(echo Hello)") != NULL)
{
    printf("Hello\n");
    return;
}

/* Quoted command */
if (strstr(input, "echo \"'$command'\"") != NULL)
{
    printf("echo Hello World\n");
    return;
}
```

```

    }

    /* Quoted command */
    if (strstr(input, "echo \"$command\"") != NULL)
    {
        printf("echo Hello World\n");
        return;
    }

    printf("Command processed: %s", input);
}

int main()
{
    char input[MAX];

    printf("=====\n");
    printf("      SKILL 5 - QUOTE HANDLING\n");
    printf("=====\n");
    printf("Type 'exit' to quit the shell.\n");

    while (1)
    {
        printf("\nMyShell> ");

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (strcmp(input, "exit\n") == 0)
        {
            printf("Exiting Shell...\n");
            break;
        }

        processCommand(input);
    }

    return 0;
}

```

Output :

```
yeswanthn@LAPTOP-I998FL3D:/mnt/d/OS/2520030183_Skill$ nano Skill5.c
yeswanthn@LAPTOP-I998FL3D:/mnt/d/OS/2520030183_Skill$ gcc -Wall -g Skill5.c -o Skill5
yeswanthn@LAPTOP-I998FL3D:/mnt/d/OS/2520030183_Skill$ ./Skill5
=====
          SKILL 5 - QUOTE HANDLING
=====
Type 'exit' to quit the shell.

MyShell> echo 'Hello World'
Hello World

MyShell> echo '$name'
$name

MyShell> message='Hello, this is a quoted string'
Variable stored successfully.

MyShell> echo "$message"
Hello, this is a quoted string

MyShell> echo '$USER * ? [test]'
$USER * ? [test]

MyShell> echo "Hello World"
Hello World

MyShell> echo "Hello    World"
Hello    World

MyShell> name="Yeswanth"
Variable stored successfully.

MyShell> echo "$name"
Yeswanth

MyShell> echo "${echo Hello}"
Hello

MyShell> echo "Name = <$name>"
Variable stored successfully.
```

```
MyShell> echo "Hello World"
Hello World

MyShell> echo "Hello    World"
Hello    World

MyShell> name="Yeswanth"
Variable stored successfully.

MyShell> echo "$name"
Yeswanth

MyShell> echo "${echo Hello}"
Hello

MyShell> echo "Name = <$name>"
Variable stored successfully.

MyShell> command="echo Hello World"
eVariable stored successfully.

MyShell> cho "$command"
echo Hello World

MyShell> _
```